

CRUD-операции с использованием Prisma

Полный Product API без ручных INSERT, SELECT, UPDATE и DELETE

Результаты занятия

- ✓ Связывать CRUD-операции с методами Prisma Client
- ✓ Получать коллекцию через `findMany()`
- ✓ Получать одну запись через `findUnique()`
- ✓ Создавать записи через `create()`
- ✓ Изменять записи через `update()`
- ✓ Удалять записи через `delete()`
- ✓ Использовать `where`, `data`, `select` и `orderBy`
- ✓ Реализовывать полный CRUD внутри Product Service
- ✓ Сохранять HTTP-логику в Controller

Учебные вопросы

- 1 CRUD в Prisma Client** Соответствие REST, CRUD и методов ORM
- 2 Create** Создание Product через `prisma.product.create()`
- 3 Read** `findMany()`, `findUnique()`, фильтрация и сортировка
- 4 Update и Delete** Изменение и удаление одной записи по уникальному условию
- 5 Полный Product Service** Переход REST API на Prisma без изменения маршрутов



Учебный вопрос 1. CRUD в Prisma Client

Каждая базовая операция получает собственный метод модели

CRUD после подключения Prisma

- PostgreSQL по-прежнему хранит данные.
- Prisma Client предоставляет методы работы с моделями.
- Product Service больше не обязан писать SQL вручную.
- HTTP-методы REST API не меняются.
- Route и Controller сохраняют прежнюю ответственность.

Главная идея

Меняется механизм доступа к данным, а не архитектура REST API.

REST → CRUD → Prisma

HTTP	CRUD	Prisma Client	Назначение
POST	Create	<code>create()</code>	Создать товар
GET	Read	<code>findMany()</code>	Получить список
GET	Read	<code>findUnique()</code>	Получить один объект
PATCH	Update	<code>update()</code>	Изменить товар
DELETE	Delete	<code>delete()</code>	Удалить товар

Пять основных методов Product

● findMany()

Возвращает массив записей Product.

● findUnique()

Ищет одну запись по id или другому уникальному полю.

● create()

Создаёт новую запись и возвращает её.

● update()

Изменяет выбранную запись и возвращает новое состояние.

● delete()

Удаляет выбранную запись и возвращает удалённый объект.

REST CRUD → методы Prisma Client



Два ключевых объекта Prisma-запроса

JAVASCRIPT

```
await prisma.product.update({  
  where: { id: 15 },  
  data: { price: 42000 }  
});
```

Что важно

- where определяет, какие записи нужно найти или изменить.
- data содержит новые данные для create() и update().
- orderBy задаёт порядок результатов.
- select ограничивает возвращаемые поля.
- include добавляет связанные объекты.

Что возвращают методы Prisma

Метод	Типичный результат
<code>findMany()</code>	Массив объектов или []
<code>findUnique()</code>	Объект или null
<code>create()</code>	Созданный объект
<code>update()</code>	Обновлённый объект
<code>delete()</code>	Удалённый объект



Учебный вопрос 2. Create

Создаём новую строку Product через объект data

Метод `create()`

- `create()` добавляет одну новую запись модели в базу данных и возвращает созданный объект.
- `data` содержит значения новой записи.
- `id` можно не передавать, если он создаётся автоматически.
- Обязательные поля должны иметь значения.
- Ограничения PostgreSQL продолжают действовать.
- Результат можно сразу отправить клиенту.

Создание Product

JAVASCRIPT

```
const product = await prisma.product.create({
  data: {
    name: "SSD 2TB",
    price: 65000,
    categoryId: 2,
    inStock: true
  }
});
```

Что важно

- `prisma.product` выбирает модель `Product`.
- `create()` означает создание одной записи.
- `data` описывает значения полей.
- `id` создаётся PostgreSQL автоматически.
- `product` получает созданный JavaScript-объект.

Create в Product Service

JAVASCRIPT

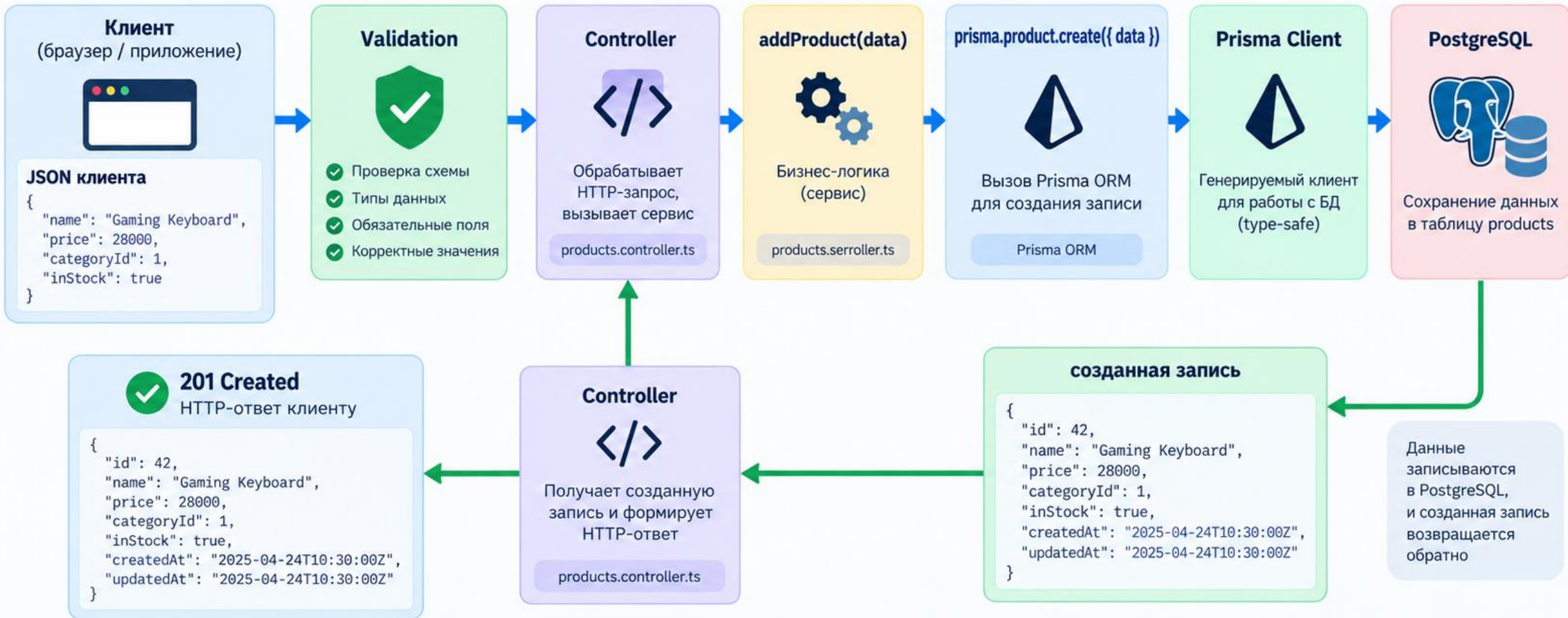
```
export async function addProduct(data) {  
  return prisma.product.create({  
    data: {  
      name: data.name,  
      price: data.price,  
      categoryId: data.categoryId,  
      inStock: data.inStock  
    }  
  });  
}
```

Что важно

- Service получает проверенные данные от Controller.
- Service не использует req и res.
- SQL INSERT вручную больше не требуется.
- Prisma возвращает созданную запись.

POST /api/products → `prisma.product.create()`

Создание товара в Express-приложении через Prisma



HTTP работает между клиентом и API, Prisma — между Service и PostgreSQL.

РАЗНЫЕ УРОВНИ.
ОДНА ЭКОСИСТЕМА. БОЛЬШИЕ ВОЗМОЖНОСТИ.

Controller создания товара

JAVASCRIPT

```
export async function createProduct(req, res, next) {
  try {
    const product = await addProduct(req.body);
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller получает HTTP body.
- Validation middleware должен проверить данные заранее.
- Service выполняет операцию Prisma.
- Controller формирует 201 Created.
- Ошибки передаются errorHandler.

Создание товара через REST API

Запрос: POST /api/products

Код • JSON

```
{  
  "name": "SSD 2TB",  
  "price": 65000,  
  "categoryId": 2,  
  "inStock": true  
}
```

Результат • JSON

```
{  
  "id": 21,  
  "name": "SSD 2TB",  
  "price": 65000,  
  "categoryId": 2,  
  "inStock": true  
}
```

Пояснение

- Prisma create() создаёт строку PostgreSQL и возвращает объект с автоматически созданным id.

Validation остаётся необходимой

- Prisma не заменяет проверку HTTP-запроса.
- name необходимо проверить до Service.
- price должен соответствовать правилам приложения.
- categoryId должен иметь правильный тип.
- inStock должен быть Boolean.
- Ограничения PostgreSQL остаются последним уровнем защиты целостности.



Учебный вопрос 3. Read

Получаем коллекции и отдельные записи

findMany(): список товаров

JAVASCRIPT

```
const products = await prisma.product.findMany({
  orderBy: {
    id: "asc"
  }
});
```

Что важно

- findMany() возвращает массив.
- Если записей нет, возвращается пустой массив.
- orderBy задаёт сортировку.
- SQL SELECT и ORDER BY вручную не пишутся.

Фильтрация через where

JAVASCRIPT

```
const products = await prisma.product.findMany({
  where: {
    price: {
      gte: 10000
    },
    inStock: true
  },
  orderBy: {
    price: "asc"
  }
});
```

Что важно

- where определяет условия отбора.
- gte означает greater than or equal.
- Несколько полей работают как совместные условия.
- orderBy сортирует найденные записи.

findUnique(): один товар

JAVASCRIPT

```
const product = await prisma.product.findUnique({
  where: {
    id: 15
  }
});
```

Что важно

- findUnique() используется с уникальным идентификатором.
- PRIMARY KEY id является уникальным.
- Найденная строка преобразуется в JavaScript-объект.
- Если записи нет, возвращается null.

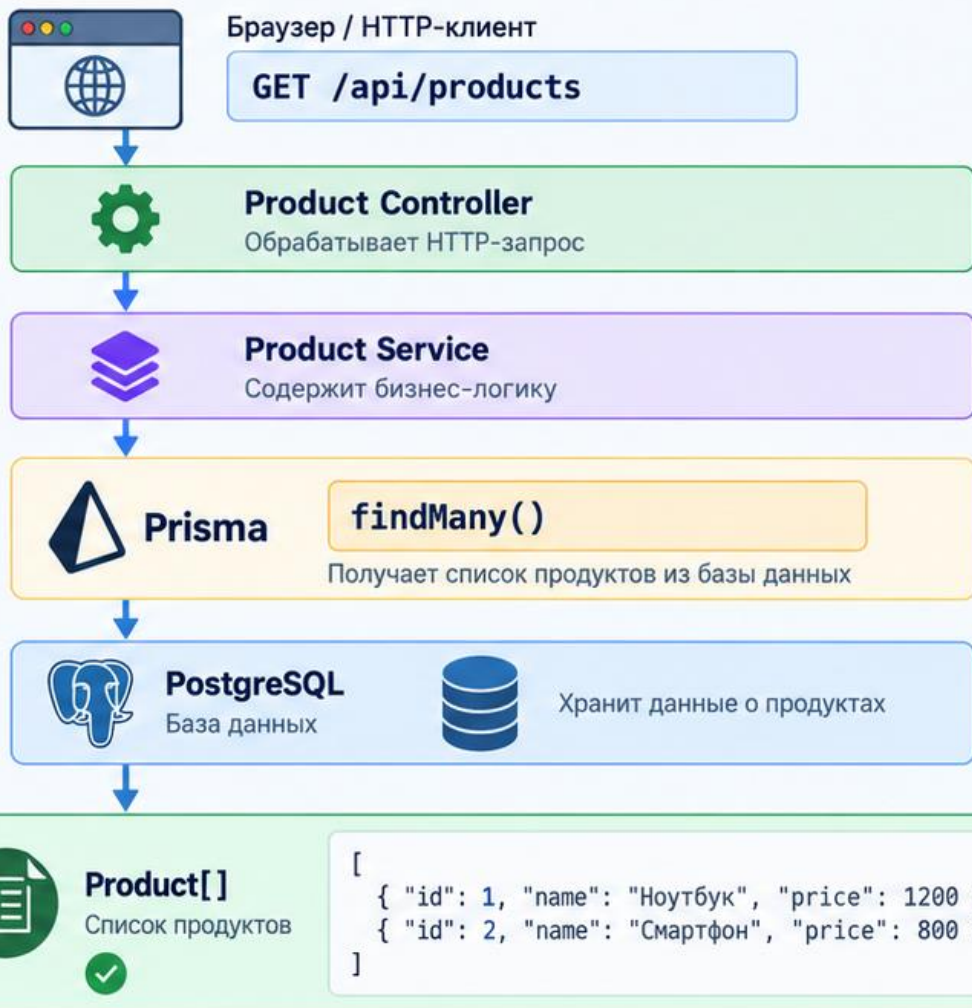
null превращается в HTTP 404

- Prisma работает с данными, а не с HTTP.
- `findUnique()` не формирует статус 404.
- Service возвращает объект или `null`.
- Controller проверяет результат.
- Если объекта нет, Controller передаёт `AppError` со статусом 404.

GET: findMany() и findUnique()

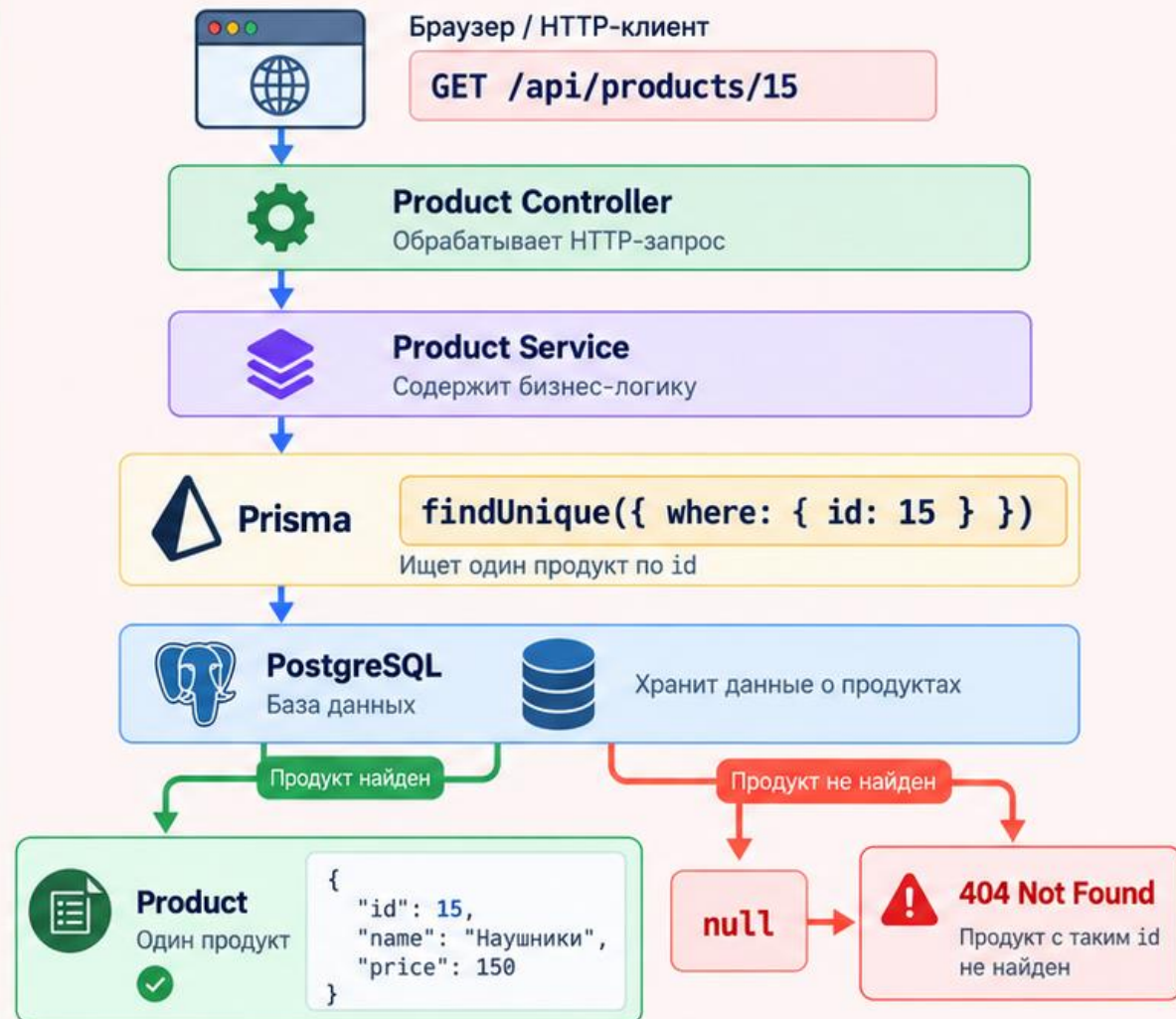
Получение списка продуктов (коллекция)

findMany() возвращает список (массив) продуктов.



Получение одного продукта по id

findUnique() возвращает один объект или null, если продукт не найден.



findMany() возвращает массив, findUnique() — один объект или null.

Выбираем только нужные поля

JAVASCRIPT

```
const products = await prisma.product.findMany({
  select: {
    id: true,
    name: true,
    price: true,
    inStock: true
  }
});
```

Что важно

- select управляет структурой результата.
- true означает, что поле нужно вернуть.
- Ненужные поля отсутствуют в объекте.
- Это полезно для формирования компактного API-ответа.

findMany() и findUnique()

findMany()

- Возвращает массив
- Поддерживает фильтрацию
- Поддерживает сортировку
- Может вернуть []
- Используется для коллекции

findUnique()

- Возвращает один объект
- Ищет по уникальному условию
- Подходит для PRIMARY KEY
- Может вернуть null
- Используется для конкретного ресурса



Учебный вопрос 4. Update и Delete

Изменяем и удаляем конкретный объект по уникальному where

Метод `update()`

- `update()` изменяет одну запись, выбранную через уникальное условие `where`, и возвращает обновлённый объект.
- `where` выбирает запись.
- `data` содержит новые значения.
- Необязательно изменять все поля.
- Метод хорошо соответствует PATCH.
- При отсутствии требуемой записи Prisma сообщает об ошибке операции.

Изменяем цену Product

JAVASCRIPT

```
const product = await prisma.product.update({
  where: {
    id: 15
  },
  data: {
    price: 42000
  }
});
```

Что важно

- where выбирает Product с id = 15.
- data содержит только изменяемое поле.
- Остальные поля сохраняются.
- Результат содержит новое состояние объекта.

PATCH и объект changes

- PATCH передаёт только изменяемые свойства.
- После validation middleware объект req.body содержит разрешённые поля.
- Service может передать эти изменения в data.
- Не переданные свойства изменять не требуется.
- Это проще прежнего SQL с COALESCE.

Update в Product Service

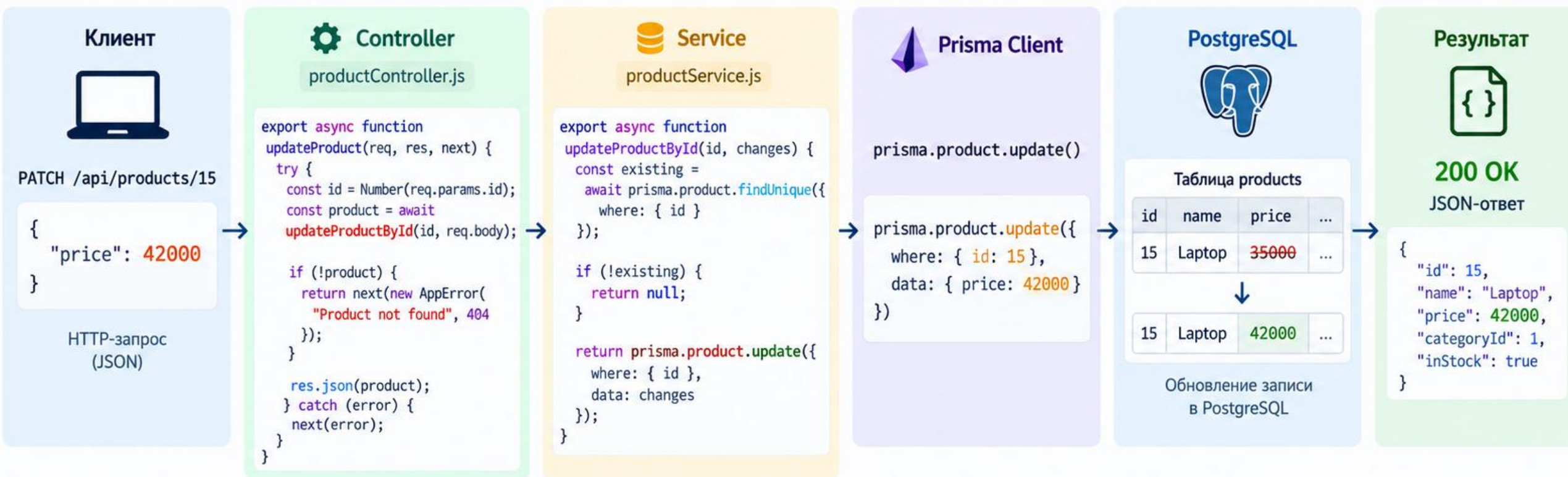
JAVASCRIPT

```
export async function updateProductById(id, changes) {
  return prisma.product.update({
    where: { id },
    data: changes
  });
}
```

Что важно

- id используется только в where.
- changes содержит поля PATCH-запроса.
- Validation должна запрещать неизвестные свойства.
- Prisma формирует UPDATE автоматически.

Частичное обновление товара через Prisma Client



Важно: PATCH обновляет только переданные поля. Остальные поля остаются без изменений.

Что делать, если Product не существует

- `findUnique()` возвращает `null`, если запись не найдена.
- `update()` ожидает существующую запись.
- Для учебного проекта можно сначала выполнить `findUnique()`.
- Если объект отсутствует, `Service` возвращает `null`.
- Если объект найден, выполняется `update()`.
- Позже можно централизованно преобразовывать ошибки Prisma в HTTP-ошибки.

Update с проверкой существования

JAVASCRIPT

```
export async function updateProductById(id, changes) {
  const existing = await prisma.product.findUnique({
    where: { id }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.update({
    where: { id },
    data: changes
  });
}
```

Что важно

- Первый запрос проверяет наличие товара.
- null совместим с уже существующей логикой Controller.
- Второй запрос изменяет запись.
- Такой вариант прост для понимания на этапе изучения CRUD.

Controller UPDATE

JAVASCRIPT

```
export async function updateProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await updateProductById(id, req);

    if (!product) {
      return next(new AppError("Product not found", 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller остаётся HTTP-слоем.
- Service скрывает детали Prisma.
- null преобразуется в 404.
- Успешный результат отправляется как JSON.

Метод delete()

- delete() удаляет одну запись, выбранную через уникальное where, и возвращает удалённый объект.
- Для Product удобно использовать id.
- После операции строка исчезает из PostgreSQL.
- Возвращённый объект можно отправить клиенту.
- Связи и FOREIGN KEY могут ограничивать удаление.
- Если обязательная запись не существует, операция завершается ошибкой.

Удаляем Product

JAVASCRIPT

```
const product = await prisma.product.delete({
  where: {
    id: 15
  }
});
```

Что важно

- where выбирает одну запись.
- id должен быть уникальным.
- Prisma выполняет DELETE в PostgreSQL.
- product содержит данные удалённого объекта.

Delete с проверкой существования

JAVASCRIPT

```
export async function deleteProductById(id) {
  const existing = await prisma.product.findUnique({
    where: { id }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.delete({
    where: { id }
  });
}
```

Что важно

- Service сначала проверяет существование товара.
- При отсутствии возвращается null.
- При наличии выполняется delete().
- Controller может сохранить прежнее поведение 404.

Удаление товара через Prisma Client

Клиент



DELETE /api/products/15

Controller

productController.js

```
export async function
deleteProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await
    deleteProductById(id);

    if (!product) {
      return next(new AppError(
        "Product not found", 404));
    };
    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Service

productService.js

```
export async function
deleteProductById(id) {
  const existing = await
    prisma.product.findUnique({
      where: { id }
    });

  if (!existing) {
    return null;
  }

  return prisma.product.delete({
    where: { id }
  });
}
```



Prisma Client

```
await prisma.product.delete({
  where: { id: 15 }
})
```

PostgreSQL

Таблица products
(до удаления)

id	name	price	...
15	Laptop	42000	...

Удаление строки
с id = 15Таблица products
(после удаления)

id	name	price	...
(нет строки с id = 15)			

Результат (успешно)



200 OK

JSON-ответ
(удалённый объект)

```
{
  "id": 15,
  "name": "Laptop",
  "price": 42000,
  "categoryId": 1,
  "inStock": true
}
```

Товар не существует

DELETE /api/products/999

```
prisma.product.findUnique({
  where: { id: 999 }
})
// возвращает null
```

return null



404 Not Found

```
{
  "error": "Product not found"
}
```



Важно: Сначала проверяем существование товара. Если Product найден — удаляем и возвращаем удалённый объект.
Если Product не найден — возвращаем 404 Not Found.

DELETE физически удаляет строку
из таблицы products в PostgreSQL.

Controller DELETE

JAVASCRIPT

```
export async function deleteProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await deleteProductById(id);

    if (!product) {
      return next(new AppError("Product not found", 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- HTTP endpoint остаётся прежним.
- Controller не вызывает Prisma напрямую.
- Service отвечает за доступ к данным.
- Удалённый объект возвращается клиенту.



Учебный вопрос 5. Полный Product Service

Все операции с моделью Product сосредоточены в одном слое

Read-операции Product Service

JAVASCRIPT

```
export async function getAllProducts() {  
  return prisma.product.findMany({  
    orderBy: { id: "asc" }  
  });  
}  
  
export async function findProductById(id) {  
  return prisma.product.findUnique({  
    where: { id }  
  });  
}
```

Что важно

- findMany() обслуживает коллекцию.
- findUnique() обслуживает один ресурс.
- Service возвращает обычные JavaScript-данные.
- Route и Controller не знают деталей ORM.

Create-операция Product Service

JAVASCRIPT

```
export async function addProduct(data) {  
  return prisma.product.create({  
    data: {  
      name: data.name,  
      price: data.price,  
      categoryId: data.categoryId,  
      inStock: data.inStock  
    }  
  });  
}
```

Что важно

- create() заменяет INSERT.
- data содержит значения нового Product.
- Результат сразу возвращается Controller.

SQL и Prisma: окончательное соответствие

Раньше	Теперь
<code>SELECT ... FROM products</code>	<code>prisma.product.findMany()</code>
<code>SELECT ... WHERE id = \$1</code>	<code>prisma.product.findUnique()</code>
<code>INSERT ... RETURNING</code>	<code>prisma.product.create()</code>
<code>UPDATE ... RETURNING</code>	<code>prisma.product.update()</code>
<code>DELETE ... RETURNING</code>	<code>prisma.product.delete()</code>

Четыре REST-операции, одна архитектура, без ручного SQL

REST API (HTTP)

POST

/api/products

Создать товар

JSON
(новый товар)

GET

/api/products

Получить список товаров

Параметры
(id)

GET

/api/products/:id

Получить товар по id

PATCH

/api/products/:id

Изменить товар

JSON
(изменения)

DELETE

/api/products/:id

Удалить товар

Параметры
(id)

Product Controller



Принимает HTTP-запросы

Проверяет данные
(Validation)Вызывает методы
сервисаФормирует HTTP-ответы
(статусы и JSON)

Product Service

Содержит бизнес-логику
Использует Prisma Client
Больше нет pool.query()

Никаких ручных SQL-запросов

addProduct(data)

getAllProducts()
findProductById(id)

updateProductById(id, changes)

deleteProductById(id)

Prisma Client

Типобезопасные запросы
к базе данныхprisma.product.create({
 data: ...
})prisma.product.findMany()
prisma.product.findUnique({
 where: { id: ... }
})prisma.product.update({
 where: { id: ... },
 data: ...
})prisma.product.delete({
 where: { id: ... }
})

PostgreSQL

Хранилище данных
(таблица products)

products

id	name	price	...
1	Laptop	42000	...
2	Mouse	3500	...
3	Monitor	28000	...
...	...	...	...



SQL исчезает из Service

Больше нет pool.query(), ручных SQL-запросов и строк с SQL. Все операции выполняются через Prisma Client.



PostgreSQL остаётся

Prisma продолжает работать с вашей существующей базой данных. Все изменения реально сохраняются в PostgreSQL.



Те же принципы REST

Архитектура приложения не меняется: Routes → Controller → Service → Database. Мы заменили только способ доступа к данным.

Prisma Studio после CRUD

BASH

```
npx prisma@7 studio
```

Что важно

- После POST новая запись появляется в Product.
- После PATCH видно изменённое значение.
- После DELETE строка исчезает.
- Studio удобно использовать как дополнительный способ проверки.
- Основной результат лабораторной всё равно проверяется через REST API.

Мини-практика: полный CRUD через Prisma

Переведите все операции Product API с ручного SQL на методы Prisma Client.

Задание

- GET /api/products → findMany()
- GET /api/products/:id → findUnique()
- POST /api/products → create()
- PATCH /api/products/:id → update()
- DELETE /api/products/:id → delete()
- Сохранить существующую validation
- Сохранить ответы 404 для отсутствующих товаров
- Проверить изменения непосредственно в PostgreSQL или Prisma Studio

Критерии проверки

- Product Service не содержит pool.query()
- Все CRUD-операции выполняются Prisma Client
- Routes не изменены
- Controllers не содержат Prisma-запросов
- POST возвращает 201
- Несуществующий Product возвращает 404
- Данные сохраняются в PostgreSQL

Контрольные вопросы

1. Какой метод Prisma соответствует Create?
2. Чем `findMany()` отличается от `findUnique()`?
3. Для чего используется `where`?
4. Для чего используется `data`?
5. Что возвращает `findUnique()`, если запись отсутствует?

6. Какой метод соответствует PATCH?
7. Что возвращает `delete()` после успешного удаления?
8. Почему Prisma не следует вызывать непосредственно из Route?
9. Зачем сохранять `validation middleware` при использовании ORM?
10. Что меняется в REST API при переходе с SQL на Prisma?

Итоги занятия

1

Prisma Client предоставляет методы CRUD для каждой модели.

2

`findMany()` возвращает коллекцию записей.

3

`findUnique()` ищет одну запись по уникальному условию.

4

`create()` добавляет новую запись.

5

`update()` изменяет выбранную запись.

6

`delete()` удаляет выбранную запись.

7

`where` определяет объект операции, а `data` содержит новые данные.

8

Validation и HTTP-обработка остаются за пределами Prisma.

9

Prisma используется внутри Service.

10

Переход с ручного SQL на ORM не требует изменения публичных REST endpoints.